

EV CHARGER DATA PLATFORM

Turning chaotic charger data into **accurate, on-time** PSC reporting.

An AI-powered pipeline that cleans, validates, reconciles, and reports on EV-charger data from third-party integrators — built end-to-end and proven on Con Edison's own territory data.

[Explore the solution](#)[See the test results](#)[View on GitHub ↗](#)

Prefer the raw numbers? [↓ Download the PSC compliance report \(CSV\)](#) · [HTML view](#)

0

RAW RECORDS PROCESSED

0

GOLDEN RECORDS OUT

0

DUPLICATES REMOVED

0

CONFLICTS RESOLVED

1

The problem

Why EV-charger data is a reporting liability today — and why it doesn't have to be.

Con Edison's EV-charger programs depend on external contractors whose data arrives through third-party integrators. It is frequently **late, incomplete, inconsistent, and unstructured** — and today, turning

it into a regulatory report is a **manual** process that strains **PSC mandates to report accurately and on time.**



Charging sessions

Structured CSV logs — but with date-format chaos, W/kW unit swaps, and missing fields.



Station registry

GeoJSON/API dumps with free-text connector names and conflicting coordinates.



Contractor reports

Unstructured email and fault digests that only a human (or an LLM) can read.

Background research

What the data itself told us — and the findings that shaped every design decision.

Before proposing a fix, we studied how the problem actually behaves in the wild: the formats, the failure modes, and the patterns that keep showing up across datasets. The goal was to design for the real world, not for a clean-room example.

Across public datasets and industry material, one pattern dominates: **the mess is in the data you already have**, not in exotic formats. Unit swaps, date chaos, free-text enums, and late-arriving files are the norm. The fix therefore had to be a **pipeline of small, testable stages** — not one clever model.



One surprising fact. In the official NYC DOT dataset — supposedly clean municipal data — our pipeline still found **4,105 duplicate records ($\approx 1.6\%$)**, **2,731 conflicting field values** no single source could resolve, and **three unresolved SAFETY maintenance events** hiding in plain sight. The cost of not looking is regulatory.

3

Ideation process

Five phases, designed to run in parallel without ever blocking each other — then merged into one pipeline.

P1 · INGESTION

Parse everything

Parsing every format and extracting structure from contractor text with an LLM.

P2 · VALIDATION

Fix what's fixable

Declarative YAML rule engine; standardizers; missing-data derivation; per-record quality scores.

P3 · RECONCILIATION

One source of truth

Merging records from different sources that describe the same charger, into one trusted answer.

P4 · REPORTING

Regulatory output

PSC report with lineage appendix; operations dashboard; safety triage; customer-experience view.

The enabling decisions

Set the rules first, then wrote the code. Agreed on the data format and how every command works — before anyone started. let five builders work in parallel on separate branches; Everyone tested with their own sample data, and then we put it all together at the end. A fifth phase (P5) wraps it all in an orchestrator with health monitoring and an integration harness.

What failed

The honest list. Every item below was found, fixed, and re-tested.

✗ **Contract-protocol deviations caught at merge review.**

Two stages emitted their final metrics line only under a flag and skipped corrupt input instead of hard-failing. The automatic checks caught both (`d33f684` , `640ee4f`) — a reminder that "works on my fixtures" is not the contract.

✗ **A config bug that looked like correct output.**

The severity synonym `urgent-safety` normalized to `MAJOR` instead of `SAFETY` , because the shorter synonym `urgent` matched first. Only hand-verifying every fixture line caught it.

✗ **A near-public privacy leak.**

Git auto-configured a commit author with a student's NYU username and wireless hostname — permanently public in a fork. Caught before push; fixed with the GitHub no-reply address.

✗ **"Never drop a record".**

A record missing a required provenance field cannot exist in the P2. Resolution: flag it, emit it anyway with a marker, and stash raw values as `raw_<field>` extras — nothing lost, nothing silently broken.

5

What worked — the solution

One command, four stages, a full audit trail — and every decision attributable.

⚠ BEFORE — RAW CONTRACTOR INPUT

This is what lands in Con Edison's inbox today. Free text. Vague location. A made-up severity.

```
← three sources, three formats, all messy →

Source A · CSV sessions "08/21/2025","19:10:54.0000000",
    "720.3","41.369","NULL"
    ← crosses midnight, W not kW, NULL fields

Source B · AFDC registry "NISSAN FAST CHR RIVERHEAD"
    ev_connector_types: "[\"CHADEMO\", \"J1772COMBO\"]"
    open_date: 1327968000000
    ← JSON inside JSON, Unix ms timestamps

Source C · Contractor report customer complaint via 311, logged 2025-06-24
    location given: "JON - Jerome 190th
        Street Municipal Parking"
    charge_box EVB-P2042308: screen dead
    routed to: J. Alvarez
    priority: urgent-safety
    ← free text, ambiguous severity
```

✓ AFTER — GOLDEN RECORD OUTPUT

Same record, same data. After 4 automated stages. Every decision attributable.

```
← one golden record, fully traced, every field

Source A · Session "2025-08-21T19:10:54-04:00"
    "0.7203", "41.369", "PAID"
    ← ISO-8601 date, kW unit, NULL → flagged

Source B · Charger "NISSAN FAST CHR RIVERHEAD"
    ev_connector_types: "J1772, CHADEMO"
    open_date: "2012-01-31"
    ← flattened, Unix timestamp → date, 4 entries

Source C · Maintenance COMPLAINT · severity: urgent
    charge_box: EVB-P2042308
    location: "JON - Jerome 190th St Municipal Parking"
    ← NLP/LLM-extracted from free text, severity tagged

merged_from audit trail preserved · 2,731 complaints
4,105 duplicates resolved · 2.9 min total run
```

1

INGESTION — THREE FORMATS IN, ONE CANONICAL FORMAT OUT

Tracing a 311 complaint from Jerome Avenue through the pipeline

Here's one real contractor report — free text about a dead screen at charge box EVB-P2042308. The ingestion stage uses **NLP and LLM extraction** to pull structured events from unstructured prose, alongside

CSV and GeoJSON format readers. Two paths: a deterministic regex engine (the default, runs offline with zero API cost) and a Claude LLM path with automatic regex fallback. All sources land in a single `canonical.jsonl` format.

258,343 → 258,519

RECORDS IN → RECORDS OUT (+176 MUNICIPAL CHARGE BOXES CREATED)

⚠ RAW INPUT

```
// Source C - Contractor free-text report
customer complaint via 311, logged 2025-06-24
location given: "JON - Jerome 190th Street
- Municipal Parking"
charge box EVB-P2042308: screen dead
routed to: J. Alvarez
priority: urgent-safety
```



✓ PARSED RECORD

```
// canonical.jsonl - typed maintenance record
{ "record_type": "maintenance",
  "event_type": "COMPLAINT",
  "severity": "SAFETY",
  "charge_box": "EVB-P2042308",
  "location": "JON - Jerome 190th St",
  "date": "2025-06-24",
  "provenance": { "source": "contractor" } }
```

Same record, now cleaned. Every fix is logged; nothing is silently dropped.

A declarative rule engine runs every record through standardizers. Dates
→ ISO-8601. Units repaired. Enums normalized. Missing values flagged in-
place. **0 records quarantined. 251,773 issues flagged.**

0

RECORDS QUARANTINED — ALL ISSUES FLAGGED IN-PLACE

~~"08/21/2025-19:10:54.0000000"~~ → 2025-08-21T19:10:54-04:00

ISO-8601 + NY offset

~~"720.3"(watts)~~ → 0.7203 kW

unit repaired

~~"NULL"~~ →  flagged · not dropped

missing field logged

~~urgent-safety~~ → SAFETY

enum normalized

~~1327968000000~~ → 2012-01-31

Unix ms → date

4 AFDC registry entries for the same charger → 1 golden record. Every merge is reversible.

Session dedup requires **same driver + same charger + same time**

window. Charger clustering uses GPS proximity. All 4,105 merged records are traceable via merged_from. **Audited: 96.5% legitimate.**

258,519 → 254,414

PRE-RECONCILIATION → GOLDEN RECORDS (4,105 MERGED)

⚠ 4 SOURCE RECORDS

station_id: 35272 — "NISSAN FAST CHR RIVERHEAD"
(J1772)

station_id: 351402 — "NISSAN FAST CHR RIVERHEAD"
(CHAdEMO)

station_id: 351403 — "NISSAN FAST CHR RIVERHEAD"
(refresh)

station_id: 351404 — "NISSAN FAST CHR RIVERHEAD"
(refresh)

All at 1800 Old Country Rd, Riverhead, NY · within 80m



✓ 1 GOLDEN RECORD

golden_id: chg-f2053989...

station_name: "NISSAN FAST CHR RIVERHEAD"

connectors: J1772, CHAdEMO

merged_from: [35272, 351402, 351403, 351404]

← full audit trail preserved

PSC compliance report with full data-lineage appendix

Every record's journey is traceable: input → fixes → golden → report. The lineage appendix proves "accurate and on time." Below: a preview of the actual PSC compliance report generated from our pipeline run.



PSC COMPLIANCE REPORT — Con Edison EV Charger Data

4,219

CHARGERS DEPLOYED

250,181

CHARGING SESSIONS

5.69 GWh

ENERGY DELIVERED

82.1%

MUNICIPAL UPTIME

89.4%

DATA COMPLETENESS

3

SAFETY EVENTS

Lineage appendix traces every record back to its source · 2.9 min end-to-end · `python -m src.platform.run --pipeline full --no-llm`

6

Testing — baseline, then the hard case

First a controlled seed dataset where we planted every known problem. Then the official NYC DOT dataset — real, uncurated, in Con Edison's own territory.

PHASE 1 · SEED DATASET

Cary, NC · 20k+ records

Synthetic messiness planted deliberately to stress every rule.

76

FIXTURE TESTS

100%

PROBLEMS CAUGHT

- ✓ **Every planted defect detected: bad dates, W/kW swaps, missing fields, stale reports, duplicate sessions. Never silently dropped a record.**

PHASE 2 · NYC DOT OFFICIAL DATA

Con Edison territory · 258k records

Real municipal garage data + NY AFDC registry + contractor reports.

26.9s

VALIDATION

2.2 min

RECONCILIATION

4,105

DUPLICATES RESOLVED

3

SAFETY EVENTS FOUND

- ✓ **Flat-memory streaming, deterministic re-runs, outputs valid against the frozen schema at ¼ million records. 298 tests pass offline.**

0

offline unit tests, all phases

0s

to validate 258k records

0 min

full reconcile

0

duplicate-billing anomalies found

7 Our recommendation

Five steps to turn this into Con Edison's reporting backbone.

- 1 Adopt the pipeline as the PSC reporting path.** Run it on every late-arriving batch: deterministic re-runs supersede prior golden records, and the lineage appendix makes "accurate and on time" auditable.
- 2 Start with validation + reporting on live data, LLM off by default.** The deterministic path already delivers most of the value at zero marginal cost.
- 3 Keep a human in the loop for SAFETY anomalies only.** Everything else — duplicates, conflicts, outliers — auto-resolves with a logged decision.
- 4 Instrument it.** Per-stage health metrics (records, duration, LLM usage) ship with every run — alerting on the anomaly feed is a small next step.
- 5 Then extend.** Contractor-portal feeds, nightly scheduling, and sharding the matching step.

What could go wrong

We named the risks so they can be managed instead of discovered.

⚠ **Model & provider drift**

LLM quality and pricing change without notice. Mitigation: pinned model config, deterministic fallback path, and usage monitoring.

⚠ **Schema drift**

A new integrator format or changed field meaning silently shifts interpretation. The proposal process exists — someone must run it.

⚠ **Late-arriving data at the deadline**

A file landing after the report window forces a restatement. The pipeline supports it; the process must allow it.

⚠ **Scale**

Now 2.2 minutes at 258k records after the audit-driven fix. Full-state rollout keeps the same architecture — the bottleneck was the matching predicate, not the pipeline.

⚠ **Overtrust in scores**

A 0.99 quality score is not ground truth. Issue codes, conflict logs, and raw-value stashes exist so analysts can always see why.

⚠ **Credential & privacy hygiene**

Keys live only in the environment; the pipeline scans every submission for them. Identifiable field media stays off public channels.

Questions for Con Edison

We built this for you — now help us understand how it fits into your reality.

- 1 Do you already use AI or NLP in your data workflows today?** If so, where — and what's working versus what isn't? We want to slot into your stack, not replace something that's already delivering.
- 2 How much of the current data process is human versus automated?** Spreadsheet reconciliation? Manual report assembly? Knowing the split helps us quantify the time this pipeline actually saves you.
- 3 How many datasets do you typically merge into one report?** We demonstrated with three — sessions, registry, contractor reports — but your reality likely involves more sources, different formats, or different cadences. Tell us what the real integration surface looks like.
- 4 What would it take to roll out a system like this?** Who needs to approve it? What does the procurement or pilot process look like? What's the right first dataset to prove value on — a single contractor feed, a single garage, a single reporting cycle?



NYU | STERN



conEdison

EV Charger Data Platform · Applied Generative AI in Business · Summer 2026

Repository:

github.com/emz68/genAIproj